디자인 패턴

2026. 4. 14 w5

미림 마이스터고

남궁성

seong.namkung@gmail.com

11. State – 상태를 클래스로 표현(if문 이동)

새로운 조건이 추가되면 메서드를 변경해야 함. 테스트도 다시 해야

```java
class Serurity {  2 usages
    void alert() {  no usages
        if(isDay()){
            // 낮이면 전화를 하고
            phoneCall();
        } else if(isNight()) {
            // 밤이면 메시지를 보낸다.
            sendMessage();
        }
    }
```

```java
class Serurity {  2 usages
    void alert() {  no usages
        if(isDay()){
            // 낮이면 전화를 하고
            phoneCall();
        } else if(isNight()) {
            // 밤이면 메시지를 보낸다.
            sendMessage();
        } else if(isHoliday()) {
            // 휴일이면 이메일을 보낸다.
            sendEmail();
        }
    }
```

새로운 조건 추가

11. State – 상태를 클래스로 표현(if문 이동)

if문(가변)을 제거하고 외부에서 객체(실행할 코드)를 주입(외부의 if문이 뭘 주입할지 결정)

```
class Serurity {  2 usages
    void alert() {  no usages
        if(isDay()){
            // 낮이면 전화를 하고
            phoneCall();
        } else if(isNight()) {
            // 밤이면 메시지를 보낸다.
            sendMessage();
        } else if(isHoliday()) {
            // 휴일이면 이메일을 보낸다.
            sendEmail();
        }
    }
```

가변

if문 제거

불변

```
class Security {  2 usages
    State state = new DayState();

    Security() {}  1 usage
    void setState(State state){  3
        this.state = state;
    }

    void alert() {  no usages
        state.action();
    }
}
```

주입받는다

호출

```
interface State {  5 usages  3 implementations
    void action();  1 usage  3 implementations
}
class DayState implements State {  2 usages
    public void action() { phoneCall(); }
    private void phoneCall() {}  1 usage
}
```

11. State – 상태를 클래스로 표현(if문 이동)

if문(가변)을 제거하고 외부에서 객체(실행할 코드)를 주입(외부의 if문이 뭘 주입할지 결정)

불변

```
class Security {  2 usages
    State state = new DayState();

    Security() {}  1 usage
    void setState(State state){  3
        this.state = state;
    }

    void alert() {  no usages
        state.action();
    }
}
```

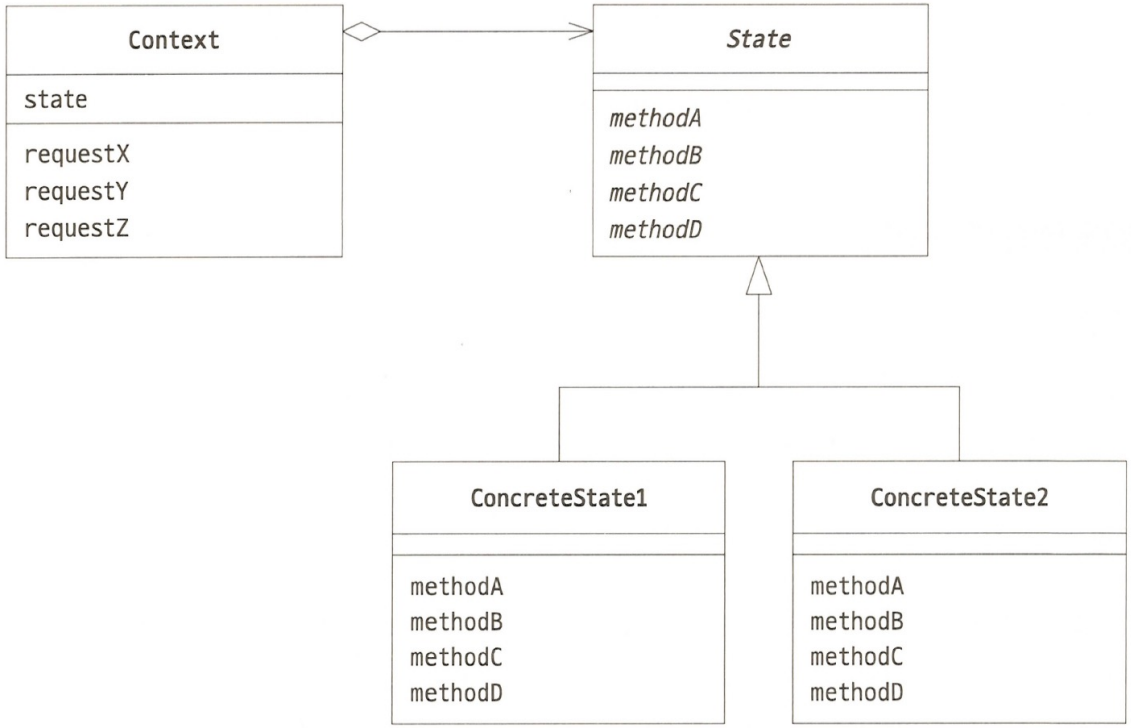
```
interface State {  5 usages  3 implementations
    void action();  1 usage  3 implementations
}
class DayState implements State {  2 usages
    public void action() { phoneCall(); }
    private void phoneCall() {}  1 usage
}
```

if문 이동

```
public static void main(String[] args) {
    Security s = new Security();

    // 조건에 따라 다른 State객체를 주입
    if(isDay()) {
        s.setState(new DayState());
    } else if(isNight()) {
        s.setState(new NightState());
    } else if(isHoliday()) {
        s.setState(new HolidayState());
    }
}
```

11. State – 상태를 클래스로 표현(if문 이동)



12. Memento – 상태를 저장

Memento패턴의 활용 예시

1. 실행취소(undo)
2. 다시 실행(redo)
3. 작업 이력 저장(history)
4. 현재 상태 저장(snapshot)

12. Memento – 상태를 저장

```java
public static void main(String[] args) {
    Gamer gamer = new Gamer( money: 100);
    Memento memento = gamer.createMemento(); // 초기 상태를 memento에 저장

    for (int i = 0; i < 10; i++) {
        try {
            gamer.play(); // game을 시작
            memento = gamer.createMemento(); // 성공하면 상태를 업데이트 한다.
        } catch (Exception e) {
            gamer.restoreMemento(memento); // 실패하면 이전 상태로 복구
        }
    }
}
```

12. Memento – 상태를 저장

```java
class Memento {  5 usages
    private int money;  2 usages
    private List<String> items;  3 usages

    public Memento(int money){  1 usage
        this.money = money;
        this.items = new ArrayList<>();
    }

    public int getMoney() { return money; }  1 usage
    void addItem(String item){  1 usage
        items.add(item);
    }
    List<String> getItems() {  1 usage
        return new ArrayList<>(items);
    }
}
```

```java
class Gamer {  3 usages
    private int money;  8 usages
    private List<String> items = new ArrayList<>();  3 usages
    private static String[] itemName = {"portion", "sword", "boots"};

    public Gamer(int money) { this.money = money; }  1 usage

    public Memento createMemento() {  2 usages
        Memento m = new Memento(money);
        for(String f: items)
            m.addItem(f);
        return m;
    }

    public void restoreMemento(Memento memento) {  1 usage
        this.money = memento.getMoney();
        this.items = memento.getItems();
    }
```

13. Observer – 상태 변화를 통보

```java
public class ObserverTest2 {
    public static void main(String[] args) {
        YouTuber youTuber = new YouTuber();
        Observer sub1 = new YouTubeSubscriber( name: "sub1");
        Observer sub2 = new YouTubeSubscriber( name: "sub2");
        Observer sub3 = new YouTubeSubscriber( name: "sub3");

        // 구독 관리는 Subject(YouTuber)가 담당
        youTuber.addObserver(sub1);
        youTuber.addObserver(sub2);
        youTuber.addObserver(sub3);

        System.out.println("youTuber.getObservers() = " + youTuber.getObservers());
        youTuber.notifyObservers("new video!!!");
    }
}
```

구독자 등록

모든 구독자의 update() 호출

13. Observer – 상태 변화를 통보

```java
class YouTuber implements Subject {  2 usages
    private List<Observer> observers = new ArrayList<>();

    public void addObserver(Observer observer) {  3 usages
        if (!observers.contains(observer))
            observers.add(observer);
    }

    public void removeObserver(Observer observer) {  no usa
        observers.remove(observer);
    }

    public void notifyObservers(String message) {  1 usage
        for (Observer o : observers)
            o.update(message);
    }

    public List<Observer> getObservers() {  1 usage
        return observers;
    }
}
```

구독자를 List에 저장

모든 구독자의 update() 호출

```java
interface Subject {  1 usage  1 implementation
    void addObserver(Observer observer);  3 u
    void removeObserver(Observer observer);
    void notifyObservers(String message);  1
}
```

```java
class YouTubeSubscriber implements Observer {  3 usages
    private String name;  3 usages

    public YouTubeSubscriber(String name) {  3 usages
        this.name = name;
    }

    public void update(String message) {  1 usage
        System.out.println("[" + name + "] " + message);
    }

    @Override
    public String toString() {
        return name;
    }
}
```

```java
interface Observer {  11 usages  1 in
    void update(String message);
}
```